Linux System Administration (E1ITA307)

Server Infrastructure Deployment Project Report

On

HOSPITAL IT DEPARTMENT SERVER MANAGEMENT

(Hospital IT Server Console)

Submitted by

SHASHANK PATEL	2411021061117
PANKAJ PANDURANG TIKE	2411021061162
ANMOL PODDAR	2411021061144
SHIKHAR SRIVASTAVA	2411021061140

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Mrs. KUSUMA J.

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Nov 2026



DEPARTMENT OF INFORMATION TECHNOLOGY
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Hospital IT Department Server Management"** submitted by Shashank Patel [2411021061117], Pankaj Pandurang Tike [2411021061162], Anmol Poddar [2411021061144] and Shikhar Srivastava [2411021061140] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Alliance University, is a bonafide work during the academic year 2026-2027.

Mrs. Kusuma J. Course Handling Faculty	Prof. Ananthanagu U Course Coordinator	Dr. Shekhar R Professor & HoD

Table of Contents

1. Introduction....4
1.1 Aim....4
1.2 Background....4
1.3 Problem Statement....4
1.4 Objectives....4
1.5 Team and Responsibilities....5
1.6 Scope of the Project....5
1.7 Organisation of the Report....5
2. Methodology....6
2.1 Tools and Technologies Used....6
2.2 Approach / Development Process....6
3. Linux System Administration – Design and Implementation....8
3.1 System Architecture....8
3.2 Linux Environment Setup....9
3.3 User and Group Management....11
3.4 File and Directory Management....13
3.5 Package and Process Management....14
3.6 Storage and File System Configuration....14
3.7 Backup and Automation....16
3.8 Security and Access Control....18
4. Results and Testing....20
4.1 Implementation Results....20
4.2 Testing and Validation....21
4.3 Observations....23
5. Conclusion and Future Scope....24
5.1 Conclusion....24
5.2 Future Scope....24
References....25
Appendices....26
Appendix A: Web Console Pages and Data....26
Appendix B: Troubleshooting....26
Appendix C: Scripts....27

1. Introduction

1.1 Aim

The main aim of this project is to design, build and verify a complete server for a hospital IT department: the Hospital IT Server Console. One server is shared by doctors, nurses and administration staff. Each group receives its own accounts and only the folder access its job needs, the data is protected by scheduled and verified backups, remote access is restricted by a firewall and a hardened SSH service, and every step is proved with real terminal output that is shown on a web console.

1.2 Background

A hospital keeps very different kinds of data on its servers. Doctors write patient records, nurses keep nursing notes, the administration office handles billing, and everyone reads the same announcements. If all staff could open every folder, a nurse could change a diagnosis or a clerk could read a patient file. If the data is not backed up, one deleted folder or one failed disk can stop the work of the whole department.

The project solves this with standard administration tools. Groups and users are created with `dscl` and `dseditgroup`, the data lives on its own disk image mounted at `/Volumes/hospital`, and the folder rules are written as Access Control Lists (ACLs) with `chmod +a`. Two `launchd` jobs run the backup every night and a disk check every hour. A `pf` firewall and an `sshd` drop-in file control how staff can connect. A script, `collect_evidence.sh`, runs 29 read-only checks and writes the real output into `public/evidence.json`, which the React web console displays.

The server was built and tested on macOS. Every Linux task of the course has a macOS equivalent (Table 3.2), and the repository also lists the Linux (Ubuntu) and Windows commands for each stage.

1.3 Problem Statement

A hospital IT department needs a single server that gives doctors, nurses and administration staff separate accounts, lets each group touch only the data its job requires, keeps the data safe against deletion and disk failure, and allows remote login only for the right people. Doing this by hand is slow, easy to get wrong and hard to prove to an auditor. The administrator needs a repeatable, scripted build and clear evidence that every rule works.

1.4 Objectives

- To create three role groups (`doctors`, `nurses`, `hosp_admin`) and six standard sample users with private home folders and no administrator rights.
- To place the hospital data on a separate 2 GB volume with four shared folders: `patient-records`, `nursing-notes`, `billing` and `announcements`.
- To give every group exactly the access in the folder access matrix, using ACLs, so that for example nurses can read but not change patient records.
- To run a backup every day at 02:00, check it with a SHA-256 checksum, keep the last seven archives and restore it with one script.
- To warn the administrator when a disk is more than 80 % full, checking every hour.
- To block incoming network traffic except SSH from the local network and to harden SSH (no root login, three tries, named users only).
- To test all six users against all four folders, audit the permissions, simulate a disaster and recover from it.
- To show the real command output of 29 checks on a web console, with a PASS or FAIL result for each stage.

1.5 Team and Responsibilities

Table 1.1: Team members and roles

Member	Register No.	Role	GitHub
Shashank Patel	2411021061117	Linux System Administrator	allenasat044-prog
Pankaj Pandurang Tike	2411021061162	Storage & Backup Administrator	pankajt25
Anmol Poddar	2411021061144	Automation & Security Engineer	ANMOL16-IT
Shikhar Srivastava	2411021061140	Testing, Documentation & Presentation Lead	FFloverSHIKHAR2006

1.6 Scope of the Project

The scripts do the real administration work: they create groups and users, the storage volume, the ACLs, the backup jobs, the firewall rules and the SSH settings. The web console only displays the evidence file; it cannot run commands or change the server. The project covers one server for one hospital department. It does not include central authentication (LDAP or Active Directory), disk quotas, encrypted or off-site backups or e-mail alerts; these are listed as future scope in Chapter 5. The Linux and Windows commands in the repository are reference procedures and were mostly not run (see section 4.2.2).

1.7 Organisation of the Report

Table 1.2: Report organisation

Chapter	Content
1. Introduction	Aim, background, problem statement, objectives and team
2. Methodology	Tools, technologies and the development process
3. Design and Implementation	Architecture, environment, users, storage, folders, automation, backup and security
4. Results and Testing	Output of the evidence run, validation table and scope of testing
5. Conclusion and Future Scope	Summary of the work and possible improvements
References and Appendices	Sources, web console pages, troubleshooting and the main scripts

2. Methodology

2.1 Tools and Technologies Used

2.1.1 Software Requirements

Table 2.1: Software used

Component	Used for
macOS 27.2.1 (build 26B131)	Server platform on which the whole build was run and tested
Bash scripts (`src/scripts/mac`)	Users, storage, automation, firewall, SSH, audit, role tests and evidence
`dscl`, `dseditgroup`	Create and manage groups, users and group membership
`hdiutil`, `diskutil`	Create and mount the 2 GB sparse disk image and enable ownership
`chmod +a` (ACLs)	Per-group folder permissions
`launchd` / `launchctl`	Schedules the nightly backup and the hourly disk check
`tar`, `shasum`, `rsync` 3.5.2	Compressed archives, SHA-256 checksums and a current rsync (Homebrew)
`pf` (`pfctl`)	Firewall that blocks incoming traffic except SSH from the local network
`sshd` (`sshd_config.d`)	Hardened remote access
Node.js 20 or newer	Builds and runs the web console
React 19, TypeScript, Vite 8, Tailwind CSS 4, React Router 7	Web console (Landing, Sign in, Evidence, FAQs)
Git and GitHub	Version control and project hosting

2.1.2 Hardware Requirements

Any Mac, laptop or virtual machine that runs macOS (or Ubuntu or Windows for the equivalent commands) is enough. About 3 GB of free disk space is needed: the data volume is a sparse image of at most 2 GB that grows only as files are added, and the backups are tiny. The evidence run was made on a Mac connected over Wi-Fi with a DHCP address on a local network.

2.1.3 Other Tools and Resources

- `public/evidence.json`: the real output of 29 checks, written by `collect_evidence.sh` and read by the web console.
- `docs/architecture.png` and `docs/architecture.svg`: the architecture diagram used in the README and on the Evidence page.
- `linux/create_users.sh`: the Linux (Ubuntu) version of the user script.
- `README.md`: stage-by-stage commands for macOS, Linux and Windows, a presentation guide and troubleshooting table.

2.2 Approach / Development Process

The project was developed in eight steps. The repository history shows the same order: stage 1 and 2, then stages 3, 4, 5 and 6, followed by the README and the web console.

Step 1: Problem identification: staff with mixed-up access, unprotected patient data, no tested backup and no proof that the rules work.

Step 2: Requirement analysis: three roles, four folders, an access matrix, scheduled backups with restore, a firewall, hardened SSH and visible evidence.

Step 3: Design: the folder access matrix, the group names (`hosp_admin` instead of `admin`, which macOS already uses for full administrators) and the architecture of four building blocks: security, identity, storage and automation.

Step 4: Stages 1 to 3: base setup, users and groups, and the storage volume with ACL folders.

Step 5: Stage 4: the backup, verify and disk-check scripts, installed and scheduled with `launchd`.

Step 6: Stage 5: the `pf` firewall, SSH hardening and the permission audit.

Step 7: Stage 6: the role tests, a simulated disaster (delete `patient-records`) and the restore script.

Step 8: Evidence and documentation: `collect_evidence.sh`, the React console, the README and this report.

3. Linux System Administration – Design and Implementation

3.1 System Architecture

The server holds four building blocks that depend on each other. Security decides who can connect, identity decides who they are, storage decides what each group can touch, and automation protects the data. The web console never runs commands: it only reads `public/evidence.json`, which `collect_evidence.sh` writes from the real state of the server. Figure 3.1 shows the complete architecture.

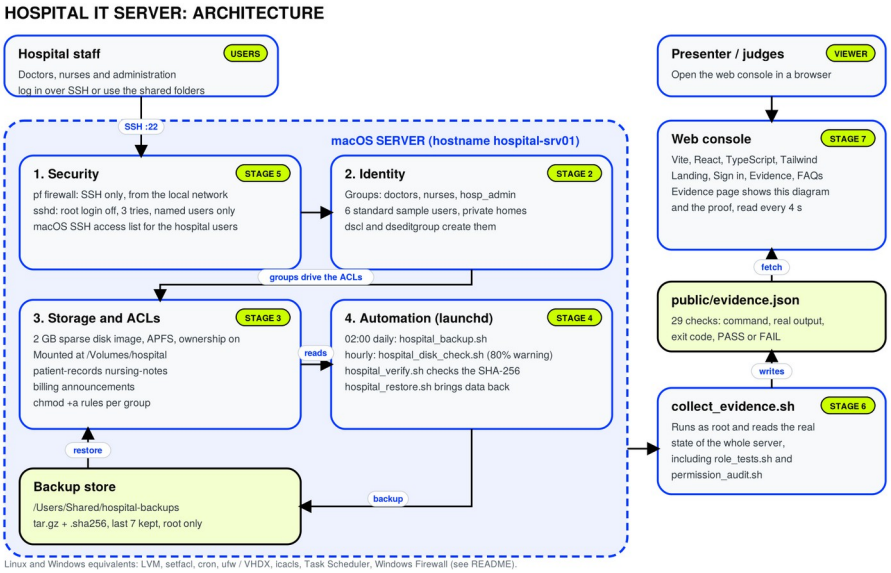


Figure 3.1: Architecture of the Hospital IT Server

Components and their roles

Table 3.1: Components and roles

Component	Role
1. Client / User	Doctors, nurses and administration staff log in over SSH or use the shared folders. The presenter opens the web console in a browser.
2. macOS Server	The Mac (design name `hospital-srv01`) that runs every script as root with `sudo` and provides `dscl`, `hdiutil`, `launchd`, `pf` and `sshd`.
3. Storage	A 2 GB sparse APFS disk image mounted at `/Volumes/hospital` with four ACL-protected folders.
4. Network	Wi-Fi interface with DHCP. `pf` allows SSH (port 22) only from the local network; the console runs on `localhost:5173`.
5. Applications / Services	Bash scripts in `src/scripts/mac`, two `launchd` jobs and the React web console.
6. Backup System	`tar.gz` archives with a `.sha256` file in `/Users/Shared/hospital-backups`; the last 7 are kept; verify and restore scripts.
7. Monitoring and Evidence	Hourly disk check, permission audit, role tests and `collect_evidence.sh`, which writes `evidence.json`.

The build is done in six stages, shown in Figure 3.2. Each script can be run again safely: existing groups and users are skipped, the disk image is reused if it exists, and every backup gets a new time-stamped name.

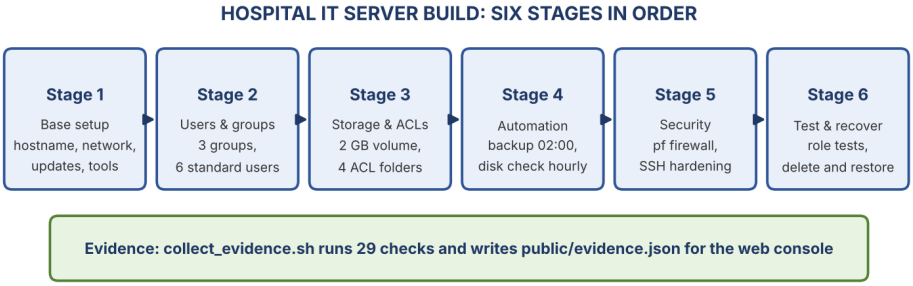


Figure 3.2: The six stages of the build

3.2 Linux Environment Setup

3.2.1 Platform and Requirements

The scripts run on macOS with the standard command line tools and must be run with `sudo` from an administrator account. Several scripts also need to know which normal account called `sudo`, because the disk image is kept in that account's home folder. Node.js 20 or newer is needed only for the web console. Table 3.2 shows how each Linux administration task of the course was done on macOS and how it is done on Windows.

Table 3.2: Linux, macOS and Windows equivalents

Task	Linux (Ubuntu)	macOS (built)	Windows
Users and groups	`useradd`, `groupadd`	`dscl`, `dseditgroup`	`New-LocalUser`, `New-LocalGroup`
Storage volume	LVM (`pvcreate`, `lvcreate`)	Sparse disk image (`hdiutil`)	VHDX with `diskpart`
Folder permissions	`setfacl`	`chmod +a`	`icacls`
Scheduling	`cron`	`launchd`	Task Scheduler
Firewall	`ufw`	`pf`	Windows Firewall
SSH hardening	`sshd_config.d`	`sshd_config.d` plus access group	`sshd_config`

3.2.2 Installation Process

The project is unpacked on the Desktop. The web console needs its npm packages once; the server scripts need nothing to be installed except the optional Homebrew `rsync`.

```
</> Bash (macOS)
cd ~/Desktop/hospital-it
brew install rsync
npm install
npm run dev
```

`npm run dev` starts the web console, which is opened at http://localhost:5173. The server stages are then run one at a time, as described in sections 3.3 to 3.8.

3.2.3 Directory Structure

```
</> Project tree
hospital-it/
├── README.md                        # stage-by-stage guide
├── docs/architecture.svg, .png      # architecture diagram
├── linux/create_users.sh            # Linux user and group script
├── public/
```

```
│   ├── architecture.svg              # diagram shown on the Evidence page
│   └── evidence.json                 # real output of 29 checks
├── src/
│   ├── App.tsx, auth.tsx, data.ts, evidence.ts
│   ├── pages/                        # Landing, Auth, Evidence, FAQ
│   ├── components/                   # Layout, Protected, ui/hero.tsx
│   └── scripts/mac/                  # 15 server scripts
│       ├── create_users.sh, remove_users.sh
│       ├── setup_storage.sh, teardown_storage.sh
│       ├── install_automation.sh, uninstall_automation.sh
│       ├── hospital_backup.sh, hospital_verify.sh, hospital_disk_check.sh
│       ├── hospital_restore.sh
│       ├── firewall.sh, harden_ssh.sh, permission_audit.sh
│       └── role_tests.sh, collect_evidence.sh
└── package.json, vite.config.ts
```

</> Server layout

```
/Volumes/hospital/                    # 2 GB sparse image, ownership on
├── patient-records/                  # ACL: doctors rw, nurses r
├── nursing-notes/                    # ACL: nurses rw, doctors r
├── billing/                          # ACL: hosp_admin rw
└── announcements/                    # ACL: all read, hosp_admin rw
/Users/Shared/hospital-backups/       # tar.gz + .sha256, root only, last 7
/usr/local/hospital/bin/              # backup, verify and disk-check scripts
/usr/local/hospital/etc/hospital.conf # settings used by the scripts
/Library/LaunchDaemons/               # com.hospital.backup, com.hospital.diskcheck
/var/log/                             # hospital-backup.log, -disk.log, -restore.log
```

Figure 3.3: Project and server directory structure

3.2.4 Network Configuration

The test Mac used DHCP on its Wi-Fi interface (address 192.168.0.102, subnet mask 255.255.255.0, router 192.168.0.1). Staff reach the server over SSH on port 22. The firewall script builds its rules for the active network interface and allows SSH only from that local network (section 3.8.1). The web console is served by Vite on port 5173.

3.2.5 User Accounts

The administrator runs the scripts from an administrator account with `sudo`. The six hospital users are different: they are standard, hidden accounts with a private home folder and no administrator rights. Their creation is described in section 3.3.

3.2.6 Required Packages

Stage 1 checks that every tool the later stages depend on exists (`dscl`, `dseditgroup`, `hdiutil`, `pfctl`, `launchctl`, `ssh`) and that a current `rsync` is installed with Homebrew. It also checks for pending operating system updates.

</> Bash (macOS)

```
softwareupdate -l
brew install rsync
which dscl dseditgroup hdiutil pfctl launchctl ssh
```

3.2.7 Scheduling with launchd

`launchd` is the native macOS scheduler and replaces `cron`. It also runs a missed job when the Mac wakes up. `install_automation.sh` copies the scripts to `/usr/local/hospital/bin`, writes `hospital.conf` and creates two property list files in `/Library/LaunchDaemons`.

```
</> com.hospital.backup.plist (excerpt)
<key>Label</key>
<string>com.hospital.backup</string>
<key>ProgramArguments</key>
<array><string>/usr/local/hospital/bin/hospital_backup.sh</string></array>
<key>StartCalendarInterval</key>
<dict><key>Hour</key><integer>2</integer>
      <key>Minute</key><integer>0</integer></dict>
```

The disk-check job (`com.hospital.diskcheck`) uses `StartInterval` 3600, which is one hour. Both jobs are loaded with `launchctl bootstrap system`.

3.3 User and Group Management

3.3.1 User and Group Creation

`create_users.sh` asks for one shared password for the sample users, creates the three groups and then the six users. A group or user that already exists is skipped, and a user id or group id that is taken by something else stops the script, so nothing foreign is overwritten.

Table 3.3: Groups and users created

Group	GID	Members (UID)	Role
`doctors`	801	`dr_asha` (801), `dr_ravi` (802)	Doctors
`nurses`	802	`nurse_meera` (803), `nurse_john` (804)	Nurses
`hosp_admin`	803	`admin_priya` (805), `admin_kiran` (806)	Administration staff

```
</> Bash (from create_users.sh)
for def in "${GROUP_DEFS[@]}"; do
  IFS='|' read -r name gid <<< "$def"
  if dscl . -read "/Groups/$name" > /dev/null 2>&1; then
    echo "Group exists: $name"
  else
    if dscl . -list /Groups PrimaryGroupID | awk -v id="$gid" '$2 == id { found = 1 } END {
      exit !found }'; then
      echo "GID $gid is already used by another group." >&2
      exit 1
    fi
    dseditgroup -o create -n . -i "$gid" -r "$name" "$name"
    echo "Group created: $name"
  fi
done
```

```
</> Bash (from create_users.sh)
    dscl . -create "/Users/$name"
    dscl . -create "/Users/$name" UserShell /bin/zsh
    dscl . -create "/Users/$name" RealName "$real"
    dscl . -create "/Users/$name" UniqueID "$uid"
    dscl . -create "/Users/$name" PrimaryGroupID 20
    dscl . -create "/Users/$name" NFSHomeDirectory "$HOME_ROOT/$name"
    dscl . -create "/Users/$name" IsHidden 1
    mkdir -p "$HOME_ROOT/$name"
    chown "$name:staff" "$HOME_ROOT/$name"
    chmod 700 "$HOME_ROOT/$name"
```

`dseditgroup -o create` makes the group with a fixed GID. For each user `dscl` sets the shell, real name, UID, primary group (`staff`, id 20) and home folder, and `IsHidden 1` keeps the account off the login screen. The home folder is created, given to the user and set to mode 700. The result is checked with:

```
</> Bash
dscl . -read /Groups/doctors GroupMembership
id nurse_meera
dseditgroup -o checkmember -m nurse_meera admin
```

3.3.2 File Ownership and Permissions

The hospital folders are owned by `root:wheel` with mode 700, so nobody except root can open them through the normal owner, group and others model. All real access is granted by ACL entries. Table 3.4 lists the two permission sets that `setup_storage.sh` uses.

Table 3.4: ACL permission sets

Level	ACL rights granted
Read and write (`rw`)	`list`, `search`, `add_file`, `add_subdirectory`, `delete_child`, `readattr`, `writeattr`, `readextattr`, `writeextattr`, `file_inherit`, `directory_inherit`
Read only (`r`)	`list`, `search`, `readattr`, `readextattr`, `file_inherit`, `directory_inherit`
None	No ACL entry for the group, so access is denied

The `file_inherit` and `directory_inherit` rights make new files and sub-folders receive the same rules, so a file saved later by a doctor is still readable by nurses.

3.3.3 Access Control

The folder access matrix is the core of the design. The ACLs are generated from it by a loop in `setup_storage.sh`. Figure 3.4 shows it, and Table 3.5 adds the number of ACL entries on each folder.

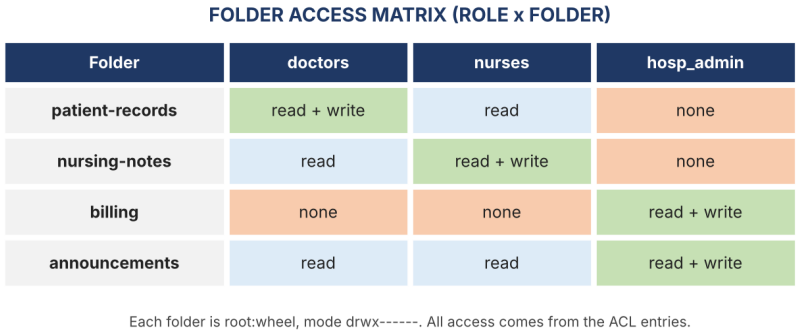


Figure 3.4: Folder access matrix

Table 3.5: Folder access and ACL entries

Folder	doctors	nurses	hosp_admin	ACL entries
`patient-records`	read and write	read	none	2
`nursing-notes`	read	read and write	none	2
`billing`	none	none	read and write	1
`announcements`	read	read	read and write	3

3.3.4 Password Management

The script reads the password twice without echo, refuses a mismatch and refuses fewer than eight characters. It sets the password for each user with `dscl . -passwd`. The password is never stored in a file. All six sample users share this one password, which is acceptable for a demonstration but not for a real hospital (section 3.8.2).

3.3.5 Linux Version

`linux/create_users.sh` does the same work on Ubuntu with `groupadd`, `useradd -m -s /bin/bash -G`, `chpasswd`, `chage -M 90 -W 7` (passwords expire after 90 days with a 7-day warning) and `chmod 700 /home/<user>`. On Linux the administration group is called `admin`, because there is no clash with a system group.

3.4 File and Directory Management

3.4.1 Directory Structure

Table 3.6: Server folders

Folder	Purpose
`/Volumes/hospital/`	Data volume, owned by root:wheel, mode 755, ownership enabled
`/Volumes/hospital/patient-records/`	Patient records written by doctors, read by nurses
`/Volumes/hospital/nursing-notes/`	Nursing notes written by nurses, read by doctors
`/Volumes/hospital/billing/`	Billing files, only for administration
`/Volumes/hospital/announcements/`	Notices written by administration, read by everyone
`/Users/Shared/hospital-backups/`	Backup archives and checksums, root:wheel, mode 700, outside the disk image
`/usr/local/hospital/`	Installed scripts (`bin`) and settings (`etc`)
`~/hospital-storage/`	The sparse disk image `hospital.sparseimage` of the administrator

3.4.2 Files Written by the Scripts

Table 3.7: Files and logs

File	Content	Kept for
`hospital-data-<time>.tar.gz`	Archive of the four folders	Last 7
`hospital-data-<time>.tar.gz.sha256`	SHA-256 checksum of the archive	With archive
`/var/log/hospital-backup.log`	Backup created, verified, old backup removed	Appended
`/var/log/hospital-disk.log`	OK or WARNING line per disk checked	Appended
`/var/log/hospital-restore.log`	Restore steps and files restored per folder	Appended
`/usr/local/hospital/etc/hospital.conf`	`VOLUME`, `IMAGE`, `BACKUP_DIR`, `FOLDERS`, `THRESHOLD=80`, `KEEP=7`	Rewritten on install
`public/evidence.json`	Output of the 29 checks	Overwritten

3.4.3 Searching and Checking Files

```bash
ls -led /Volumes/hospital/*
ls -l /Users/Shared/hospital-backups
sudo tail -n 6 /var/log/hospital-backup.log
sudo -u nurse_meera touch /Volumes/hospital/patient-records/test.txt
echo 'Patient 002 admitted' | sudo -u dr_asha tee
  /Volumes/hospital/patient-records/patient002.txt
sudo -u nurse_meera cat /Volumes/hospital/patient-records/patient002.txt
```

The first `touch` is refused with “Permission denied”, because nurses have read-only access. The doctor can write the record and the nurse can read it back.

3.5 Package and Process Management

3.5.1 Package Management

Table 3.8: Tools checked and npm scripts

Item	Purpose
`dscl`, `dseditgroup`, `hdiutil`, `pfctl`, `launchctl`, `ssh`	Tools that stage 1 checks with `which`
Homebrew `rsync` 3.5.2	Current rsync for backups (macOS ships an old one)
`npm run dev`	Starts the web console (Vite) on http://localhost:5173
`npm run build`	Type-checks with `tsc -b` and builds the console with Vite
`npm run lint`	Runs `oxlint` on the source
`npm run preview`	Previews the production build

3.5.2 Process Management

The two background jobs are managed with `launchctl`. The administrator can list them, start one immediately and read its log.

```bash
sudo launchctl list | grep hospital
sudo launchctl kickstart -k system/com.hospital.backup
sudo cat /var/log/hospital-backup.log
sudo /usr/local/hospital/bin/hospital_verify.sh
sudo /usr/local/hospital/bin/hospital_disk_check.sh 1
```

3.5.3 Service Management and Logs

Each script writes time-stamped lines to its own log in `/var/log`, and `launchd` keeps the standard output of each job in `hospital-backup.launchd.log` and `hospital-disk.launchd.log`. SSH is switched on with `sudo systemsetup -setremotelogin on` and its effective settings are read with `sshd -T`.

3.6 Storage and File System Configuration

3.6.1 The Data Volume

On macOS there is no LVM, so the data lives on a separate volume that can be mounted, unmounted and resized: a 2 GB sparse disk image. “Sparse” means the file on disk grows only when data is written. `setup_storage.sh` creates it, mounts it, switches ownership on and builds the folders.

```
</> Bash (from setup_storage.sh)
if [ -f "$IMAGE" ]; then
  echo "Disk image exists: $IMAGE"
else
  hdiutil create -size 2g -type SPARSE -fs APFS -volname hospital "$IMAGE_DIR/hospital"
  echo "Disk image created: $IMAGE"
fi

if mount | grep -q " on $VOLUME "; then
  echo "Already mounted: $VOLUME"
else
  hdiutil attach "$IMAGE"
fi

diskutil enableOwnership "$VOLUME"
chown root:wheel "$VOLUME"
chmod 755 "$VOLUME"
```

Table 3.9: Storage settings

Item	Value
Image	`~/hospital-storage/hospital.sparseimage`, 2 GB, APFS, volume name `hospital`
Mount point	`/Volumes/hospital`
Ownership	Enabled with `diskutil enableOwnership`, so owners and ACLs are honoured
Volume root	`root:wheel`, mode 755
Data folders	`root:wheel`, mode 700, ACLs removed with `chmod -N` and then added with `chmod +a`

The image is unmounted after a restart. Running `setup_storage.sh` again mounts it, and the backup script also mounts it automatically when it finds it missing.

3.6.2 ACL Folders

After the volume is ready, the script loops over the four folders. For each folder it removes old ACLs and adds one `chmod +a` rule per group that has access, using the matrix in Table 3.5.

```
</> Bash (from setup_storage.sh)
for def in "${FOLDERS[@]}"; do
  IFS='|' read -r folder d n a <<< "$def"
  levels=("$d" "$n" "$a")
  dir="$VOLUME/$folder"
  mkdir -p "$dir"
  chown root:wheel "$dir"
  chmod 700 "$dir"
  chmod -N "$dir"
  i=0
  for g in "${ROLE_GROUPS[@]}"; do
    case "${levels[$i]}" in
      rw) chmod +a "group:$g allow $RW" "$dir" ;;
      r) chmod +a "group:$g allow $R" "$dir" ;;
    esac
    i=$((i + 1))
  done
```

3.6.3 Disk Monitoring

`hospital_disk_check.sh` runs every hour. It reads the use of the hospital volume and of the backup storage with `df` and compares it with the limit in `hospital.conf` (80 %). It prints OK or WARNING, writes the line to `/var/log/hospital-disk.log` and exits with 1 when any warning was raised. A limit can be given as an argument, which is used in the demonstration to force a warning.

</> Bash (from hospital_disk_check.sh)

```
check() {
  label="$1"
  path="$2"
  if [ ! -d "$path" ]; then
    log "WARNING: $label not available at $path"
    STATUS=1
    return
  fi
  used="$(df -k "$path" | awk 'NR == 2 { gsub("%", "", $5); print $5 }')"
```

3.6.4 Recommended Linux Layout (LVM, reference)

The Linux version of the design uses a loop-backed LVM volume so that it can be resized. **Note:** these commands are a reference procedure from the README and were not run in the project tests.

</> Bash (Ubuntu)

```
sudo truncate -s 2G /srv/hospital.img
LOOP=$(sudo losetup --find --show /srv/hospital.img)
sudo pvcreate "$LOOP" && sudo vgcreate hospital_vg "$LOOP"
sudo lvcreate -n data -l 100%FREE hospital_vg
sudo mkfs.ext4 /dev/hospital_vg/data
sudo mkdir -p /hospital && sudo mount /dev/hospital_vg/data /hospital
sudo setfacl -m g:doctors:rwx,g:nurses:r-x /hospital/patient-records
sudo lvextend -r -L +1G /dev/hospital_vg/data
```

3.7 Backup and Automation

3.7.1 Backup Strategy

Every night at 02:00 `hospital_backup.sh` archives the four folders into `hospital-data-YYYYMMDD-HHMMSS.tar.gz` in `/Users/Shared/hospital-backups`, which is outside the disk image and readable only by root. It stores a SHA-256 checksum file beside the archive, verifies the archive at once, and keeps only the seven newest archives.

3.7.2 Backup and Verification

</> Bash (from hospital_backup.sh)

```
STAMP="$(date +%Y%m%d-%H%M%S)"
NAME="hospital-data-$STAMP.tar.gz"
ARCHIVE="$BACKUP_DIR/$NAME"

# shellcheck disable=SC2086
COPYFILE_DISABLE=1 tar -czf "$ARCHIVE" -C "$VOLUME" $FOLDERS
(cd "$BACKUP_DIR" && shasum -a 256 "$NAME" > "$NAME.sha256")
log "Backup created: $ARCHIVE"

if "$BIN_DIR/hospital_verify.sh" "$ARCHIVE" > /dev/null; then
  log "Backup verified: $NAME"
```

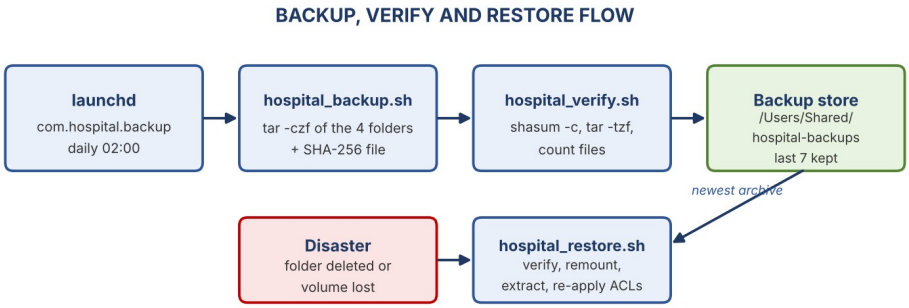


Figure 3.5: Backup, verify and restore flow

`hospital_verify.sh` checks the checksum with `shasum -a 256 -c`, tests that the archive can be read with `tar -tzf` and prints the number of files in it. If verification fails, the backup job stops with an error and keeps the old archives.

```
</> Bash (from hospital_verify.sh)
cd "$(dirname "$ARCHIVE")"
NAME="$(basename "$ARCHIVE")"

shasum -a 256 -c "$NAME.sha256"
tar -tzf "$NAME" > /dev/null
echo "Archive readable: $NAME"
echo "Files in archive: $(tar -tzf "$NAME" | grep -vc '/$' || true)"
```

3.7.3 Recovery Procedure

`hospital_restore.sh` takes the newest archive (or one given as an argument) and verifies it before anything is changed. It then runs `setup_storage.sh` to make sure the volume is mounted, extracts the archive, runs `setup_storage.sh` again to re-apply the folder permissions and ACLs, and logs how many files each folder received. The recovery demonstration deletes `patient-records` on purpose and restores it.

```
</> Bash (disaster and recovery)
sudo launchctl kickstart -k system/com.hospital.backup
sudo rm -rf /Volumes/hospital/patient-records
sudo ls /Volumes/hospital
sudo bash src/scripts/mac/hospital_restore.sh
sudo ls -led /Volumes/hospital/patient-records
sudo -u nurse_meera cat /Volumes/hospital/patient-records/patient002.txt
```

3.7.4 Built-in Automation

Table 3.10: Automation shipped with the project

Mechanism	What it does
`launchd` job `com.hospital.backup`	Runs the backup every day at 02:00
`launchd` job `com.hospital.diskcheck`	Runs the disk check every hour (80 % limit)
Automatic verification	Every backup is checked against its SHA-256 before it is accepted
Automatic rotation	Backups beyond the newest seven are removed and logged
Automatic mount	The backup script attaches the disk image if the volume is not mounted
Repeatable scripts	Existing users, groups and images are skipped; ACLs are rebuilt from the matrix

Mechanism	What it does
Undo scripts	`remove_users.sh`, `teardown_storage.sh`, `uninstall_automation.sh`, `firewall.sh off`, `harden_ssh.sh remove`

3.8 Security and Access Control

Table 3.11: Security controls

Control	Detail
Least privilege	Each group gets only the rights in the access matrix; the sample users are standard users, never administrators
Private homes	Home folders of the users are mode 700
No world-writable items	The audit finds none on `/Volumes/hospital`
Firewall (`pf`)	Inbound traffic is blocked except SSH from the local network
SSH hardening	Root login off, empty passwords off, three tries, 60 second login window, named users only
Protected backups	Backup folder is `root:wheel` mode 700 and every archive has a checksum
Permission audit	`permission_audit.sh` checks 19 items and prints PASS or FAIL

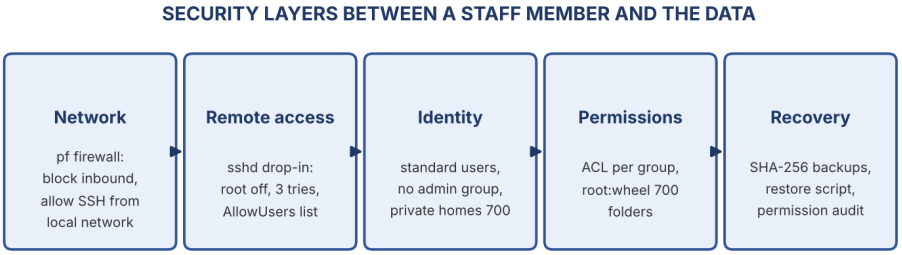


Figure 3.6: Security layers

3.8.1 Firewall and SSH

`firewall.sh` finds the active interface and builds a `pf` anchor called `com.hospital`. The rules are checked with `pfctl -nf` before they are loaded. After enabling the firewall the script tests the network with `ping` to the router or `curl`, and rolls everything back if the test fails, so a mistake cannot lock the administrator out.

Table 3.12: Firewall rules (interface en0)

Rule	Effect
`pass quick on lo0 all`	Local traffic is always allowed
`block in on en0 all`	All other incoming traffic is blocked
`pass out on en0 all keep state`	Outgoing traffic is allowed
`pass in ... from (en0:network) ... port 22`	SSH only from the local network
`pass in` ICMP, ICMPv6, DHCP (67 to 68), mDNS (5353)	Ping, IPv6 neighbour discovery, address and name services

`harden_ssh.sh apply` writes `/etc/ssh/sshd_config.d/100-hospital.conf`, tests it with `sshd -t` and removes it again if it is invalid. It also adds the allowed users to the macOS group `com.apple.access_ssh`, because macOS keeps its own SSH access list in addition to `AllowUsers`.

</> 100-hospital.conf

```
PermitRootLogin no
PermitEmptyPasswords no
MaxAuthTries 3
LoginGraceTime 60
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
AllowUsers shashank dr_asha dr_ravi nurse_meera nurse_john admin_priya admin_kiran
```

3.8.2 Known Limits

- The six sample users share one password. It is typed at run time and never stored, but a real deployment needs one password or key per person.
- The `pf` rules and the disk image mount do not survive a restart; `firewall.sh on` and `setup_storage.sh` must be run again.
- Remote Login makes SSH reachable from the local network while it is on; the undo steps switch it off.
- Backups are on the same Mac as the data and are not encrypted; an off-site, encrypted copy is recommended.
- Restored files get their ACLs from the folder; ACLs set by hand on single files are not restored.
- The console sign-in stores a SHA-256 hash of the password in the browser’s local storage. It only hides the Evidence page from casual visitors and is not a security control.

4. Results and Testing

4.1 Implementation Results

The repository contains the output of a real evidence run. `collect_evidence.sh` was run with `sudo` on 11 November 2026 on the test Mac (macOS 27.2.1, user `shashank`) and ran 29 checks. All 29 passed. Figure 4.1 shows the result for each stage and Table 4.1 summarises the findings.

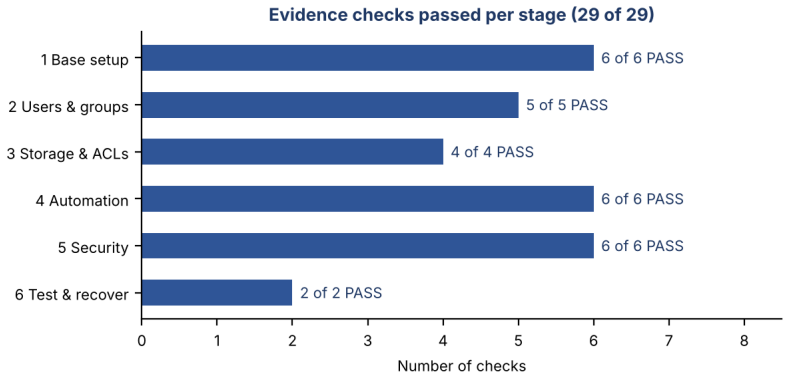


Figure 4.1: Evidence checks passed per stage

Table 4.1: Results of the evidence run

Item	Result
Groups	doctors: dr_asha, dr_ravi; nurses: nurse_meera, nurse_john; hosp_admin: admin_priya, admin_kiran
Administrator rights	All six users report “is NOT a member of admin”
Home folders	`drwx------` for `dr_asha`, `nurse_meera` and `admin_priya`
Storage volume	`hospital`, APFS, mounted at `/Volumes/hospital`, ownership enabled
ACLs	`patient-records` 2 entries, `nursing-notes` 2, `billing` 1, `announcements` 3
Access denied	Nurse cannot write patient records; admin cannot read them; doctor cannot open billing
Access granted	Nurse lists `patient001.txt`; doctor writes to patient records
Scheduled jobs	`com.hospital.backup` and `com.hospital.diskcheck` listed by `launchctl`
Backups	4 archives of 375 to 381 bytes, each with a 103-byte `.sha256`; newest `hospital-data-20261111-020005.tar.gz`
Integrity	Newest archive: checksum OK, readable, 3 files in the archive
Disk usage	Hospital volume 1 % full, backup storage 34 % full (limit 80 %); with the limit set to 1 % both lines became WARNING
Firewall	`pf` enabled, rules in anchor `com.hospital` as in Table 3.12
SSH	`PermitRootLogin no`, `MaxAuthTries 3`, `LoginGraceTime 60`, `X11Forwarding no`, `AllowUsers` for 7 accounts
Permission audit	19 PASS lines, “All checks passed.”
Role tests	24 of 24 cells correct, “All role checks passed.”
Restore	Backup verified, volume ready, files restored, ACLs re-applied; 1, 1, 1 and 0 files in the four folders

The role test script tries to read and write every folder as every user and compares the result with the matrix. Figure 4.2 shows its output exactly as it was captured.

```
</> $ sudo bash src/scripts/mac/role_tests.sh
user         patient-records   nursing-notes     billing          announcements
dr_asha      rw ok             r ok              - ok             r ok
dr_ravi      rw ok             r ok              - ok             r ok
nurse_meera  r ok              rw ok             - ok             r ok
nurse_john   r ok              rw ok             - ok             r ok
admin_priya  - ok              - ok              rw ok            rw ok
admin_kiran  - ok              - ok              rw ok            rw ok

All role checks passed.
```

Figure 4.2: Role test result

The nurse and the administrator are refused where the matrix says so. The command output below was captured by the access checks.

```
</> Evidence (s3-denied and s3-allowed)
nurse_meera writes to patient-records:
touch: /Volumes/hospital/patient-records/.evidence_probe: Permission denied
admin_priya lists patient-records:
ls: /Volumes/hospital/patient-records: Permission denied
dr_asha lists billing:
ls: /Volumes/hospital/billing: Permission denied
nurse_meera lists patient-records:
patient001.txt
dr_asha writes to patient-records:
write allowed
```

Figure 4.3: Access denied and access granted

```
</> $ sudo bash src/scripts/mac/permission_audit.sh
PASS: ownership is enabled on /Volumes/hospital
PASS: patient-records is drwx------ root:wheel
PASS: patient-records has 2 ACL entries
PASS: nursing-notes is drwx------ root:wheel
PASS: nursing-notes has 2 ACL entries
PASS: billing is drwx------ root:wheel
PASS: billing has 1 ACL entries
PASS: announcements is drwx------ root:wheel
PASS: announcements has 3 ACL entries
PASS: no world-writable items on /Volumes/hospital
PASS: dr_asha is a standard user   (same line for the other five users)
PASS: backup folder is drwx------ root:wheel
PASS: SSH root login is disabled
PASS: firewall is enabled with the hospital rules

All checks passed.
```

Figure 4.4: Permission audit (shortened: six user lines shown as one)

4.2 Testing and Validation

Table 4.2: Testing and validation

Test case	Expected result	Actual result	Status
Base setup	All tools present; system checked for updates	Six tools found; “No new software available”; rsync 3.5.2	Pass

Test case	Expected result	Actual result	Status
Groups and users	Three groups with two members each	doctors, nurses and hosp_admin each list two members	Pass
No admin rights	No sample user in `admin`	All six “NOT a member of admin”	Pass
Private homes	Users cannot read each other’s files	Home folders `drwx------`	Pass
Storage volume	Separate APFS volume with ownership on	`/Volumes/hospital`, APFS, owners enabled	Pass
ACL rules	Each folder grants exactly the matrix	ACL lines match Table 3.5	Pass
Access denied / granted	Nurse read only; admin no records; doctor no billing	Three “Permission denied”; read and write allowed where expected	Pass
Scheduled jobs	Backup and disk-check jobs loaded	Both listed by `launchctl`	Pass
Backup and checksum	Archive plus `.sha256`, verified	4 archives with checksums; newest verified OK	Pass
Disk warning	Warning when the limit is lowered to 1 %	Two WARNING lines, exit code 1	Pass
Firewall	Block inbound, allow SSH from local network	Rules listed in anchor `com.hospital`	Pass
SSH settings	Root off, 3 tries, 60 s, named users	All settings shown by `sshd -T`	Pass
Permission audit	All items PASS	19 PASS, “All checks passed.”	Pass
Role tests	24 cells match the matrix	24 of 24 ok	Pass
Delete and restore	Deleted folder returns with permissions	Backup verified, files restored, ACLs re-applied	Pass

4.2.1 Reproducing the Tests

```bash
sudo bash src/scripts/mac/role_tests.sh
sudo bash src/scripts/mac/permission_audit.sh
sudo /usr/local/hospital/bin/hospital_verify.sh
sudo bash src/scripts/mac/collect_evidence.sh
npm run build
```

4.2.2 Honest Scope of Testing

All server checks were run on one Mac, so the numbers describe a small test system: the hospital volume was 1 % full and each backup holds only a few sample files (375 to 381 bytes). During the evidence run the Mac still carried its own computer name; the name `hospital-srv01` is the design name and is set only for the presentation. The README states that the macOS stages were run on a real Mac and checked stage by stage, that `linux/create_users.sh` was run and the backup and disk-check scripts were tested on Linux, and that the other Linux commands and all Windows commands are reference procedures that were not run. The evidence collector’s JSON writer and its pass or fail logic were tested with sample checks. The web console was not tested with an automated browser test.

4.2.3 Testing Summary

The tests show that the six users receive exactly the access in the matrix, that none of them is an administrator, that a deleted folder can be restored from a verified backup together with its permissions, that the disk check raises a warning, and that the firewall and SSH settings are in force. All 29 evidence checks passed.

4.3 Observations

- The ACL design gives different rights on one folder to different groups, which plain owner, group and others permissions cannot do.
- Naming the group `hosp_admin` avoided making hospital staff administrators of the whole Mac, because macOS already has a group called `admin`.
- A checksum is stored with every backup and checked immediately and again before a restore, so a damaged archive is never restored.
- The restore script re-applies the ACLs, so a restored folder is protected as before.
- Two problems were found through the logs and fixed in the scripts: macOS keeps its own SSH access list that blocked standard users, and the first SSH status check missed a setting because of upper and lower case.
- The `launchd` list shows the disk-check job with last exit status 1. The script exits with 1 whenever it raises a warning, so this is consistent with the demonstration run that lowers the limit; the check made during the evidence run returned OK.
- Limitations: one shared sample password, a disk image and firewall that need to be restored after a restart, backups on the same machine and no encryption.

5. Conclusion and Future Scope

5.1 Conclusion

The Hospital IT Server Console turns the requirements of a hospital IT department into a repeatable build. In six stages it creates three role groups and six standard users, puts the data on a separate volume, protects four folders with ACLs generated from one access matrix, schedules a daily backup that is verified with SHA-256, warns when a disk is nearly full, restricts the network with a firewall and hardens SSH. A script records 29 checks with their real output and a React console displays them with a PASS or FAIL result for every stage.

The project gave us practical experience in system administration: users and groups, ownership and ACLs, storage volumes, scheduling, backup and recovery, firewalls and secure remote access. We learned that least privilege is easiest to keep when permissions are generated from one table, that a backup is only useful after its restore has been tested, and that evidence from the real system is more convincing than a description.

5.2 Future Scope

- Central login with LDAP or Open Directory instead of local accounts, and one password or SSH key per person.
- Encrypted backups and an encrypted data volume, because the data is patient information.
- A second, off-site copy of the backups, with a restore test on the other machine.
- Automatic mounting of the disk image and loading of the firewall at start-up.
- E-mail or chat alerts when the disk check warns or a backup fails verification.
- Access logs and audit reports that show who opened which folder.
- Testing the Linux and Windows procedures and adding them to the evidence collector.
- A small API so that the console shows live results instead of the last saved run, and an automated browser test for the console.

References

1. Hospital IT Server Console project source code: https://github.com/allenasat044-prog/OrangeCats

2. Apple manual pages: dscl(1), dseditgroup(8), hdiutil(1), diskutil(8), chmod(1) (ACLs), launchctl(1), launchd.plist(5), pfctl(8), tar(1), shasum(1). Apple Developer Documentation: https://developer.apple.com/documentation/

3. OpenBSD manual page sshd_config(5): https://man.openbsd.org/sshd_config

4. OpenBSD PF User’s Guide: https://www.openbsd.org/faq/pf/

5. Linux manual pages: useradd(8), groupadd(8), chpasswd(8), chage(1), setfacl(1), lvcreate(8). https://man7.org/linux/man-pages/

6. Node.js documentation: https://nodejs.org/docs

7. React, Tailwind CSS and Vite documentation: https://react.dev, https://tailwindcss.com, https://vite.dev

8. Course material, Linux System Administration (E1ITA307), Alliance School of Advanced Computing, Alliance University.

Appendices

Appendix A: Web Console Pages and Data

The web console has four pages. It reads `public/evidence.json` every four seconds and does not change the server. The Evidence page is shown only after signing in; accounts are stored in the browser only.

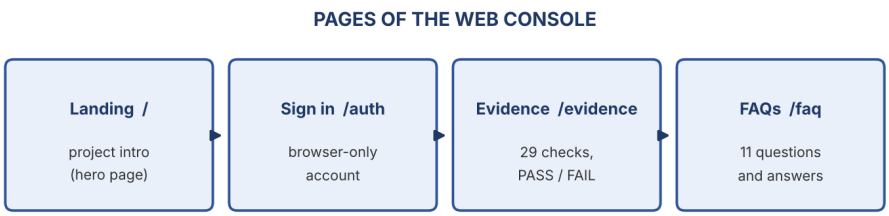


Figure A1: Pages of the web console

Table A.1: Pages and what they show

Page	Shows
Landing (`/`)	Introduction to the project
Sign in (`/auth`)	Sign in or create a console account (name, e-mail, password of at least 6 characters)
Evidence (`/evidence`)	Architecture diagram, the command to capture evidence, a summary such as “29 of 29 checks passed”, and for each stage the checks with command, real output, exit code and PASS or FAIL
FAQs (`/faq`)	Eleven questions and answers about the project

Table A.2: Main fields of evidence.json

Field	Content
`generatedAt`, `host`, `macos`, `user`	Time of capture, computer name, macOS version and user
`summary`	`total`, `passed` and `failed` counts
`checks[].id`, `stage`, `title`	Check identifier, stage number 1 to 6 and title
`checks[].proves`	One sentence on what the check proves
`checks[].command`, `output`, `exitCode`	The command that was run, its real output and its exit code
`checks[].status`	`pass` or `fail`

Appendix B: Troubleshooting

Table B.1: Common problems

Symptom	Cause	Fix
`/Volumes/hospital` missing or storage checks fail	The disk image is unmounted after a restart	`sudo bash src/scripts/mac/setup_storage.sh`
Internet stops after the firewall step	`pf` rules blocked something	`sudo bash src/scripts/mac/firewall.sh off`

Symptom	Cause	Fix
Firewall status shows Disabled	`pf` rules are cleared by a restart	`sudo bash src/scripts/mac/firewall.sh on`
SSH closes right after the password	User not on the macOS SSH access list, or password mistyped	`sudo bash src/scripts/mac/harden_ssh.sh apply`
Evidence page shows no evidence	The collector has not been run	`sudo bash src/scripts/mac/collect_evidence.sh`
Evidence shows FAIL for the restore log	No restore has been run yet	Run the delete and restore step once
`npm run dev` says the port is busy	An old dev server is running	Close it or stop the process shown by `lsof -i :5173`
`systemsetup` refuses to change Remote Login	Terminal lacks Full Disk Access	Turn on Remote Login in System Settings, General, Sharing

To undo the whole build, run the following in order:

</> Bash (undo)

```
sudo bash src/scripts/mac/uninstall_automation.sh
sudo bash src/scripts/mac/firewall.sh off
sudo bash src/scripts/mac/harden_ssh.sh remove
sudo systemsetup -setremotelogin off
sudo bash src/scripts/mac/teardown_storage.sh
sudo bash src/scripts/mac/remove_users.sh
sudo rm -rf /Users/Shared/hospital-backups
```

Appendix C: Scripts

The main scripts are listed below exactly as they are in `src/scripts/mac` (long lines are wrapped for printing). The remaining scripts (`install_automation.sh`, `permission_audit.sh`, `collect_evidence.sh` and the undo scripts) are in the repository.

</> src/scripts/mac/create_users.sh

```
#!/bin/bash
set -eu

if [ "$(id -u)" -ne 0 ]; then
  echo "Run this script with sudo." >&2
  exit 1
fi

HOME_ROOT="${HOME_ROOT:-/Users}"

GROUP_DEFS=(
  "doctors|801"
  "nurses|802"
  "hosp_admin|803"
)

USER_DEFS=(
  "dr_asha|801|doctors|Dr Asha"
  "dr_ravi|802|doctors|Dr Ravi"
  "nurse_meera|803|nurses|Nurse Meera"
  "nurse_john|804|nurses|Nurse John"
  "admin_priya|805|hosp_admin|Admin Priya"
  "admin_kiran|806|hosp_admin|Admin Kiran"
```

```
)

printf "Password for the sample users (minimum 8 characters): "
read -rs PASS
echo
printf "Confirm password: "
read -rs PASS2
echo

if [ "$PASS" != "$PASS2" ]; then
  echo "Passwords do not match." >&2
  exit 1
fi
if [ "${#PASS}" -lt 8 ]; then
  echo "Password must be at least 8 characters." >&2
  exit 1
fi

for def in "${GROUP_DEFS[@]}"; do
  IFS='|' read -r name gid <<< "$def"
  if dscl . -read "/Groups/$name" > /dev/null 2>&1; then
    echo "Group exists: $name"
  else
    if dscl . -list /Groups PrimaryGroupID | awk -v id="$gid" '$2 == id { found = 1 } END {
      exit !found }'; then
      echo "GID $gid is already used by another group." >&2
      exit 1
    fi
    dseditgroup -o create -n . -i "$gid" -r "$name" "$name"
    echo "Group created: $name"
  fi
done

for def in "${USER_DEFS[@]}"; do
  IFS='|' read -r name uid group real <<< "$def"
  if dscl . -read "/Users/$name" > /dev/null 2>&1; then
    echo "User exists: $name"
  else
    if dscl . -list /Users UniqueID | awk -v id="$uid" '$2 == id { found = 1 } END { exit
      !found }'; then
      echo "UID $uid is already used by another user." >&2
      exit 1
    fi
    dscl . -create "/Users/$name"
    dscl . -create "/Users/$name" UserShell /bin/zsh
    dscl . -create "/Users/$name" RealName "$real"
    dscl . -create "/Users/$name" UniqueID "$uid"
    dscl . -create "/Users/$name" PrimaryGroupID 20
    dscl . -create "/Users/$name" NFSHomeDirectory "$HOME_ROOT/$name"
    dscl . -create "/Users/$name" IsHidden 1
    mkdir -p "$HOME_ROOT/$name"
    chown "$name:staff" "$HOME_ROOT/$name"
    chmod 700 "$HOME_ROOT/$name"
    echo "User created: $name"
  fi
  dscl . -passwd "/Users/$name" "$PASS"
  dseditgroup -o edit -a "$name" -t user "$group"
  echo "Added $name to $group"
```

```
done

echo
echo "Group membership:"
for def in "${GROUP_DEFS[@]}"; do
  IFS='|' read -r name gid <<< "$def"
  echo "$name: $(dscl . -read "/Groups/$name" GroupMembership 2> /dev/null | sed
    's/^GroupMembership: *//')"
done
```

</> src/scripts/mac/setup_storage.sh

```
#!/bin/bash
set -eu

if [ "$(id -u)" -ne 0 ]; then
  echo "Run this script with sudo." >&2
  exit 1
fi

if [ -z "${SUDO_USER:-}" ]; then
  echo "Run this script with sudo from your normal account." >&2
  exit 1
fi

VOLUME="${VOLUME:-/Volumes/hospital}"
OWNER_HOME="$(dscl . -read "/Users/$SUDO_USER" NFSHomeDirectory | awk '{ print $2 }')"
IMAGE_DIR="$OWNER_HOME/hospital-storage"
IMAGE="$IMAGE_DIR/hospital.sparseimage"

RW="list,search,add_file,add_subdirectory,delete_child,readattr,writeattr,readextattr,writee
  xtattr,file_inherit,directory_inherit"
R="list,search,readattr,readextattr,file_inherit,directory_inherit"

ROLE_GROUPS=(doctors nurses hosp_admin)

FOLDERS=(
  "patient-records|rw|r|none"
  "nursing-notes|r|rw|none"
  "billing|none|none|rw"
  "announcements|r|r|rw"
)

mkdir -p "$IMAGE_DIR"

if [ -f "$IMAGE" ]; then
  echo "Disk image exists: $IMAGE"
else
  hdiutil create -size 2g -type SPARSE -fs APFS -volname hospital "$IMAGE_DIR/hospital"
  echo "Disk image created: $IMAGE"
fi

if mount | grep -q " on $VOLUME "; then
  echo "Already mounted: $VOLUME"
else
  hdiutil attach "$IMAGE"
fi

diskutil enableOwnership "$VOLUME"
chown root:wheel "$VOLUME"
```

```
chmod 755 "$VOLUME"

for def in "${FOLDERS[@]}"; do
  IFS='|' read -r folder d n a <<< "$def"
  levels=("$d" "$n" "$a")
  dir="$VOLUME/$folder"
  mkdir -p "$dir"
  chown root:wheel "$dir"
  chmod 700 "$dir"
  chmod -N "$dir"
  i=0
  for g in "${ROLE_GROUPS[@]}"; do
    case "${levels[$i]}" in
      rw) chmod +a "group:$g allow $RW" "$dir" ;;
      r) chmod +a "group:$g allow $R" "$dir" ;;
    esac
    i=$((i + 1))
  done
  echo "Folder ready: $dir"
done

echo
ls -led "$VOLUME"/*
```

</> src/scripts/mac/hospital_backup.sh

```
#!/bin/bash
set -eu

CONF="${HOSPITAL_CONF:-/usr/local/hospital/etc/hospital.conf}"
. "$CONF"

LOG="${HOSPITAL_LOG_DIR:-/var/log}/hospital-backup.log"
BIN_DIR="$(cd "$(dirname "$0")" && pwd)"

log() {
  echo "$(date '+%Y-%m-%d %H:%M:%S') $*" | tee -a "$LOG"
}

if ! mount | grep -q " on $VOLUME "; then
  log "Volume not mounted, attaching $IMAGE"
  if ! hdiutil attach "$IMAGE" > /dev/null 2>&1; then
    log "ERROR: could not attach $IMAGE"
    exit 1
  fi
  diskutil enableOwnership "$VOLUME" > /dev/null 2>&1 || true
fi

mkdir -p "$BACKUP_DIR"

STAMP="$(date +%Y%m%d-%H%M%S)"
NAME="hospital-data-$STAMP.tar.gz"
ARCHIVE="$BACKUP_DIR/$NAME"

# shellcheck disable=SC2086
COPYFILE_DISABLE=1 tar -czf "$ARCHIVE" -C "$VOLUME" $FOLDERS
(cd "$BACKUP_DIR" && shasum -a 256 "$NAME" > "$NAME.sha256")
log "Backup created: $ARCHIVE"

if "$BIN_DIR/hospital_verify.sh" "$ARCHIVE" > /dev/null; then
```

```
  log "Backup verified: $NAME"
else
  log "ERROR: verification failed for $NAME"
  exit 1
fi

ls -1t "$BACKUP_DIR"/hospital-data-*.tar.gz 2> /dev/null | tail -n +$((KEEP + 1)) | while
  read -r old; do
  rm -f "$old" "$old.sha256"
  log "Old backup removed: $(basename "$old")"
done
```

</> src/scripts/mac/hospital_verify.sh

```
#!/bin/bash
set -eu

CONF="${HOSPITAL_CONF:-/usr/local/hospital/etc/hospital.conf}"
. "$CONF"

ARCHIVE="${1:-}"
if [ -z "$ARCHIVE" ]; then
  ARCHIVE="$(ls -1t "$BACKUP_DIR"/hospital-data-*.tar.gz 2> /dev/null | head -n 1 || true)"
fi

if [ -z "$ARCHIVE" ] || [ ! -f "$ARCHIVE" ]; then
  echo "No backup found." >&2
  exit 1
fi

cd "$(dirname "$ARCHIVE")"
NAME="$(basename "$ARCHIVE")"

shasum -a 256 -c "$NAME.sha256"
tar -tzf "$NAME" > /dev/null
echo "Archive readable: $NAME"
echo "Files in archive: $(tar -tzf "$NAME" | grep -vc '/$' || true)"
```

</> src/scripts/mac/hospital_restore.sh

```
#!/bin/bash
set -eu

if [ "$(id -u)" -ne 0 ]; then
  echo "Run this script with sudo." >&2
  exit 1
fi

PREFIX="${PREFIX:-/usr/local/hospital}"
CONF="${HOSPITAL_CONF:-$PREFIX/etc/hospital.conf}"
. "$CONF"

DIR="$(cd "$(dirname "$0")" && pwd)"
LOG="${HOSPITAL_LOG_DIR:-/var/log}/hospital-restore.log"

log() {
  echo "$(date '+%Y-%m-%d %H:%M:%S') $*" | tee -a "$LOG"
}

ARCHIVE="${1:-}"
if [ -z "$ARCHIVE" ]; then
  ARCHIVE="$(ls -1t "$BACKUP_DIR"/hospital-data-*.tar.gz 2> /dev/null | head -n 1 || true)"
```

```
fi

if [ -z "$ARCHIVE" ] || [ ! -f "$ARCHIVE" ]; then
  log "ERROR: no backup found in $BACKUP_DIR"
  exit 1
fi

if ! "$PREFIX/bin/hospital_verify.sh" "$ARCHIVE" > /dev/null; then
  log "ERROR: backup failed verification, nothing was restored: $ARCHIVE"
  exit 1
fi
log "Backup verified: $(basename "$ARCHIVE")"

if ! mount | grep -q " on $VOLUME "; then
  log "Volume not mounted, mounting it first"
fi
bash "$DIR/setup_storage.sh" > /dev/null
log "Volume ready: $VOLUME"

tar -xzf "$ARCHIVE" -C "$VOLUME"
log "Files restored from $(basename "$ARCHIVE")"

bash "$DIR/setup_storage.sh" > /dev/null
log "Folder permissions and ACLs re-applied"

for folder in $FOLDERS; do
  count="$(find "$VOLUME/$folder" -type f 2> /dev/null | wc -l | tr -d ' ')"
  log "Restored $folder: $count files"
done
```

</> src/scripts/mac/hospital_disk_check.sh

```
#!/bin/bash
set -eu

CONF="${HOSPITAL_CONF:-/usr/local/hospital/etc/hospital.conf}"
. "$CONF"

LIMIT="${1:-$THRESHOLD}"
LOG="${HOSPITAL_LOG_DIR:-/var/log}/hospital-disk.log"
STATUS=0

log() {
  echo "$(date '+%Y-%m-%d %H:%M:%S') $*" | tee -a "$LOG"
}

check() {
  label="$1"
  path="$2"
  if [ ! -d "$path" ]; then
    log "WARNING: $label not available at $path"
    STATUS=1
    return
  fi
  used="$(df -k "$path" | awk 'NR == 2 { gsub("%", "", $5); print $5 }')"
  if [ "$used" -ge "$LIMIT" ]; then
    log "WARNING: $label is ${used}% full (limit ${LIMIT}%)"
    STATUS=1
  else
    log "OK: $label is ${used}% full (limit ${LIMIT}%)"
```

```
  fi
}

check "hospital volume" "$VOLUME"
check "backup storage" "$BACKUP_DIR"

exit "$STATUS"
```

</> src/scripts/mac/firewall.sh

```
#!/bin/bash
set -eu

if [ "$(id -u)" -ne 0 ]; then
  echo "Run this script with sudo." >&2
  exit 1
fi

ACTION="${1:-status}"
PREFIX="${PREFIX:-/usr/local/hospital}"
ANCHOR_FILE="/etc/pf.anchors/com.hospital"
MAIN_CONF="/etc/pf.hospital.conf"
MARKER="$PREFIX/etc/pf_was_disabled"

pf_enabled() {
  pfctl -s info 2> /dev/null | grep -q '^Status: Enabled'
}

build() {
  IF="$(route -n get default 2> /dev/null | awk '/interface:/ { print $2 }')"
  GATEWAY="$(route -n get default 2> /dev/null | awk '/gateway:/ { print $2 }')"
  if [ -z "$IF" ] || [ -z "$GATEWAY" ]; then
    echo "Could not find the active network interface." >&2
    exit 1
  fi
  mkdir -p /etc/pf.anchors
  cat > "$ANCHOR_FILE" << RULES
pass quick on lo0 all
block in on $IF all
pass out on $IF all keep state
pass in on $IF proto tcp from ($IF:network) to any port 22 keep state
pass in on $IF proto icmp all keep state
pass in on $IF inet6 proto icmp6 all keep state
pass in on $IF proto udp from any port 67 to any port 68 keep state
pass in on $IF proto udp from any to any port 5353 keep state
RULES
  cp /etc/pf.conf "$MAIN_CONF"
  {
    echo 'anchor "com.hospital"'
    echo "load anchor \"com.hospital\" from \"$ANCHOR_FILE\""
  } >> "$MAIN_CONF"
  pfctl -nf "$MAIN_CONF"
}

rollback() {
  echo "$1" >&2
  pfctl -f /etc/pf.conf > /dev/null 2>&1 || true
  if [ -f "$MARKER" ]; then
    pfctl -d > /dev/null 2>&1 || true
    rm -f "$MARKER"
```

```
  fi
  echo "Firewall rules were rolled back." >&2
  exit 1
}

case "$ACTION" in
  check)
    build
    echo "Rules are valid for interface $IF:"
    cat "$ANCHOR_FILE"
    ;;
  on)
    build
    mkdir -p "$PREFIX/etc"
    if pf_enabled; then
      rm -f "$MARKER"
    else
      touch "$MARKER"
    fi
    pfctl -f "$MAIN_CONF" 2> /dev/null
    pfctl -e > /dev/null 2>&1 || true
    if ping -c 2 -t 4 "$GATEWAY" > /dev/null 2>&1 || curl -s -o /dev/null --max-time 6
      https://www.apple.com; then
      echo "Firewall is on. Network check passed on $IF."
    else
      rollback "Network check failed after enabling the firewall."
    fi
    ;;
  off)
    pfctl -f /etc/pf.conf > /dev/null 2>&1 || true
    if [ -f "$MARKER" ]; then
      pfctl -d > /dev/null 2>&1 || true
      rm -f "$MARKER"
    fi
    rm -f "$ANCHOR_FILE" "$MAIN_CONF"
    echo "Firewall rules removed."
    ;;
  status)
    pfctl -s info 2> /dev/null | head -n 2
    echo
    pfctl -a com.hospital -s rules 2> /dev/null || true
    ;;
  *)
    echo "Usage: sudo bash firewall.sh check|on|off|status" >&2
    exit 1
    ;;
esac
```

</> src/scripts/mac/harden_ssh.sh

```
#!/bin/bash
set -eu

if [ "$(id -u)" -ne 0 ]; then
  echo "Run this script with sudo." >&2
  exit 1
fi

if [ -z "${SUDO_USER:-}" ]; then
  echo "Run this script with sudo from your normal account." >&2
```

```
  exit 1
fi

ACTION="${1:-status}"
DROPIN="/etc/ssh/sshd_config.d/100-hospital.conf"
ALLOWED="$SUDO_USER dr_asha dr_ravi nurse_meera nurse_john admin_priya admin_kiran"

case "$ACTION" in
  apply)
    if ! grep -q '^Include /etc/ssh/sshd_config.d/' /etc/ssh/sshd_config; then
      echo "This macOS sshd_config does not include /etc/ssh/sshd_config.d. Stopping." >&2
      exit 1
    fi
    ssh-keygen -A > /dev/null 2>&1 || true
    mkdir -p /etc/ssh/sshd_config.d
    cat > "$DROPIN" << CONF
PermitRootLogin no
PermitEmptyPasswords no
MaxAuthTries 3
LoginGraceTime 60
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
AllowUsers $ALLOWED
CONF
    chown root:wheel "$DROPIN"
    chmod 644 "$DROPIN"
    if /usr/sbin/sshd -t; then
      echo "SSH hardening applied."
      echo "Allowed users: $ALLOWED"
      if dscl . -read /Groups/com.apple.access_ssh > /dev/null 2>&1; then
        for u in $ALLOWED; do
          dseditgroup -o edit -a "$u" -t user com.apple.access_ssh
        done
        echo "Allowed users added to the macOS SSH access group."
      fi
    else
      rm -f "$DROPIN"
      echo "The SSH configuration was invalid and has been removed." >&2
      exit 1
    fi
    ;;
  remove)
    rm -f "$DROPIN"
    if dscl . -read /Groups/com.apple.access_ssh > /dev/null 2>&1; then
      for u in $ALLOWED; do
        dseditgroup -o edit -d "$u" -t user com.apple.access_ssh > /dev/null 2>&1 || true
      done
    fi
    echo "SSH hardening removed."
    ;;
  status)
    out="$(/usr/sbin/sshd -T -C user=root,host=localhost,addr=127.0.0.1 2>&1 || true)"
    if ! echo "$out" | grep -i -E
      '^(permitrootlogin|permitemptypasswords|maxauthtries|logingracetime|x11forwarding|clie
      ntaliveinterval|clientalivecountmax|allowusers) '; then
      echo "sshd -T did not list the settings. Its output was:"
      echo "$out" | head -n 5
```

```
      echo
      echo "Config file $DROPIN:"
      cat "$DROPIN" 2> /dev/null || echo "(missing)"
    fi
    ;;
  *)
    echo "Usage: sudo bash harden_ssh.sh apply|remove|status" >&2
    exit 1
    ;;
esac
```

</> src/scripts/mac/role_tests.sh

```
#!/bin/bash
set -u

if [ "$(id -u)" -ne 0 ]; then
  echo "Run this script with sudo." >&2
  exit 1
fi

VOLUME="${VOLUME:-/Volumes/hospital}"
FOLDERS=(patient-records nursing-notes billing announcements)

EXPECT=(
  "dr_asha|rw|r|-|r"
  "dr_ravi|rw|r|-|r"
  "nurse_meera|r|rw|-|r"
  "nurse_john|r|rw|-|r"
  "admin_priya|-|-|rw|rw"
  "admin_kiran|-|-|rw|rw"
)

access() {
  user="$1"
  dir="$2"
  can_read=0
  can_write=0
  if sudo -u "$user" ls "$dir" > /dev/null 2>&1; then
    can_read=1
  fi
  probe="$dir/.role_test_$user"
  if sudo -u "$user" touch "$probe" > /dev/null 2>&1; then
    can_write=1
    sudo -u "$user" rm -f "$probe" > /dev/null 2>&1 || true
  fi
  rm -f "$probe"
  if [ "$can_read" = 1 ] && [ "$can_write" = 1 ]; then
    echo "rw"
  elif [ "$can_read" = 1 ]; then
    echo "r"
  elif [ "$can_write" = 0 ]; then
    echo "-"
  else
    echo "?"
  fi
}

BAD=0
printf '%-13s' "user"
```

```
for f in "${FOLDERS[@]}"; do
  printf '%-18s' "$f"
done
echo

for def in "${EXPECT[@]}"; do
  IFS='|' read -r user e1 e2 e3 e4 <<< "$def"
  expected=("$e1" "$e2" "$e3" "$e4")
  printf '%-13s' "$user"
  i=0
  for f in "${FOLDERS[@]}"; do
    actual="$(access "$user" "$VOLUME/$f")"
    if [ "$actual" = "${expected[$i]}" ]; then
      printf '%-18s' "$actual ok"
    else
      printf '%-18s' "$actual WRONG"
      BAD=$((BAD + 1))
    fi
    i=$((i + 1))
  done
  echo
done

echo
if [ "$BAD" -eq 0 ]; then
  echo "All role checks passed."
else
  echo "$BAD role check(s) failed."
  exit 1
fi
```